

XRPL Validator — Daily Engineering Report • 2026-04-05

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **30 commits** (xrpl-validator 30)

Engineering changes

- **feat: tx verification engine, consensus Phase A, FFI scaffolding** (741b4e2 , xrpl-validator) Three layers of validator independence work, delivered together: 1. Tx verification engine (deprecated per FFI pivot, retained as scratchpad) - crates/xrpl-node/src/tx_engine.rs: independently verifies transaction effects against rippled's metadata for all 53+ XRPL tx types - Deep verification for Payment (XRP direct), EscrowCreate, PayChan* - Generic fee-verification with pre-state fallback for complex types ...
- **feat(ffi): first real libxrpl call — parse mainnet Payment tx** (8e7a6de , xrpl-validator) Milestone 2A: STTx deserialization works end-to-end.
- **feat(ffi): first successful xrpl::preflight() call on real mainnet tx** (9ac92d5 , xrpl-validator) MILESTONE 2B: Full libxrpl preflight pipeline runs end-to-end via FFI.
- **feat(ffi): xrpl::apply() SUCCESS on real mainnet Payment** (2c9c10e , xrpl-validator) 🎯 MILESTONE 2C: Full transaction engine runs end-to-end through FFI.
- **feat(ffi): live demo script for visible FFI pipeline** (2ddeefe , xrpl-validator) Runs test_shim in a loop, shows live stats: - Stage counters (parse / preflight / apply) with bars - Throughput: iter/sec, calls/sec - Latency: p50/p99/avg per full cycle - Last tx result summary
- **feat(ffi): extract state mutations after apply() — full round trip** (67eb437 , xrpl-validator) 🎯 MILESTONE 2D: Rust can now get back the exact SLE changes from libxrpl's apply() and commit them to RocksDB. Full round trip complete.
- **feat(ffi): Rust FFI bindings — crate xrpl-ffi with full round trip tests** (eee986d , xrpl-validator) 🎯 MILESTONE 2E: Rust can now call libxrpl's tx engine end-to-end.
- **feat(ffi): integrate xrpl-ffi into validator (feature-gated)** (d8da1f8 , xrpl-validator) Validator can now delegate tx application to libxrpl via the Rust FFI bindings. Feature-gated so default builds are unaffected.
- **feat(ffi): push live mainnet tx through libxrpl in real-time** (1d7e2eb , xrpl-validator) The ffi_test server now fetches every new mainnet ledger from rippled and runs each tx through libxrpl's parse + preflight via our FFI.
- **feat(ffi): full apply() on live mainnet tx with RPC-fetched state** (7d0acd7 , xrpl-validator) Every few seconds, we sample ~3 txs per new mainnet ledger and run them through libxrpl's FULL apply() via FFI. SLE lookups happen synchronously via rippled's ledger_entry RPC (pre-ledger state at seq-1).
- **feat(ffi): 100% mainnet agreement — libxrpl apply in full** (0f7682f , xrpl-validator) Three root causes fixed to hit 100% tx-result agreement with mainnet on live feed, up from 55%: 1. Overlay tombstones

(LayeredProvider) Deleted SLEs were `remove()` -ed from overlay, so the next lookup fell through to RPC which returned the stale *pre-ledger* version. Now we insert `None` as a tombstone so the fallback is bypassed. Killed OfferCancel/tefBAD_LEDGER (36 → 0). 55% → 82%.

- **feat(ffl): production observability — Prometheus, Grafana, SLOs** (00b54ca , xrpl-validator) Ripple-grade metrics stack around the 100% agreement guarantee: - OpenMetrics /metrics endpoint on port 3778 exposing: * `xrpl_ffl_mainnet_agreement_ratio` (gauge, SLO target = 1.0) * `xrpl_ffl_apply_total{result=tesSUCCESS|tec_claimed|diverged}` * `xrpl_ffl_apply_duration_seconds` histogram (p50/p95/p99) * `xrpl_ffl_apply_ter_total{ter}`, `xrpl_ffl_apply_by_type_total{tx_type}` * `xrpl_ffl_diverged_total{tx_type,ter}` * ...
- **fix(ffl): retry + failover against rippled overload** (9993061 , xrpl-validator) Root cause of 97.26% agreement drop: rippled under memory/load pressure returns plain-text "Server is overloaded" responses (not JSON errors). RpcProvider was parsing these as "no node_binary found" and returning None, which libxrpl interpreted as "this SLE doesn't exist" → `terNO_ACCOUNT` for accounts that definitely exist on mainnet.
- **refactor(validator): tx_engine → FfiVerifier (libxrpl via FFI)** (5c0eef0 , xrpl-validator) Replace the 1164-line pure-Rust `tx_engine` verifier with `FfiVerifier`, which replays each ledger's tx set through libxrpl (rippled's C++ engine) and tracks mainnet agreement via the same infrastructure `ffi_test` uses. Deletes `tx_engine.rs` outright.
- **feat(ffl): Stage 1b — RocksDB-backed SLE provider** (b1aad92 , xrpl-validator) Add `OverlayedDbProvider`: 3-tier lookup (in-ledger overlay → rocksdb snapshot of the validator's own state → RpcProvider fallback). Dropping RPC load from ~10 calls/tx to ~0-1 calls/tx once the validator's DB is populated.
- **fix(ffl): OwnedSnapshot eliminates verify race condition** (0b275be , xrpl-validator) The bug: Stage 1b's `OverlayedDbProvider` took a rocksdb snapshot inside `spawn_blocking` — which could run AFTER `ws_sync's` `process_ledger` wrote post-ledger state to the DB. FFI verify then saw POST-state as PRE-state, tried to insert already-existing entries into directories, and libxrpl's `ApplyView` hit `LogicError("dirInsert: double insertion")` → `std::abort()` → process crash.
- **fix(sync): bulk_sync rpc_failover was ignoring XRPL_RPC_URL override** (8d1d941 , xrpl-validator) `rpc_failover()` iterated HARDCODED `RPC_ENDPOINTS` (localhost → `xrplcluster` → `s1.ripple.com`). On the validator host (no local rippled), this means every worker falls over to public endpoints that don't serve `ledger_data`, producing "DONE: 0 objects". The `XRPL_RPC_URL` env var only affected the initial ledger query, never the worker threads.
- **fix(ffl): disable DB snapshot during catchup — use pure RPC path** (c2f8f6a , xrpl-validator)
- **fix(rpc): rpc_request() was ignoring XRPL_RPC_URL env override — used hardcoded localhost** (1fc4d55 , xrpl-validator)
- **refactor(dashboard): update watch_engine.py for integrated FFI validator** (e21113b , xrpl-validator)
- FFI stats now read from validator's /api/engine (ffi_verifier key) instead of separate sidecar at :3778 - Removed deprecated `tx_engine` section (deleted in code) - Removed "Current Round" and "Top Tx Types" (tied to old engine) - FFI section shows: agreement %, apply count, types, TERs, divergences - Added "State Hash" section with match/mismatch/consecutive counts - `FFI_BASE` env var no longer needed (stats come ...)
- **feat(ui): new validator dashboard designed with Google Stitch** (9613958 , xrpl-validator) Replaces the old `validator.html` with a production monitoring dashboard generated via Stitch AI. Dark theme, Bloomberg-terminal aesthetic.
- **feat(ui): wire live data into dashboard — 500ms fetch from /api/engine** (6bcb8ff , xrpl-validator)
- **feat(ui): live tx-type bars with color-coded fills replacing static chart** (80eb182 , xrpl-validator)

- **feat(ui): per-ledger tx bars (reset each ledger) + sync progress indicator** (`afdbc36` , `xrpl-validator`)
- **fix(ui): remove /api/status dependency, infer sync from FFI state** (`ec154d0` , `xrpl-validator`)
- **feat(ui): show all tx types grayed out, light up when active in current ledger** (`12b4bf8` , `xrpl-validator`)
- **feat(ui): state lifecycle indicator (sync → build → match) + live object count + round tx stats** (`c190092` , `xrpl-validator`)
- **feat(ui): descriptive labels + explanations for FFI engine stats** (`6f21057` , `xrpl-validator`)
- **feat(ui): add WIP notice — dashboard in development, not all links functional** (`77082a0` , `xrpl-validator`)

Docs and operations

- **docs(ffi): clearer demo — status banner + honest labels** (`a76625e` , `xrpl-validator`)

Appendix — commits

Time	Repo	Commit	Subject
11:56	<code>xrpl-validator</code>	<code>741b4e2</code>	feat: tx verification engine, consensus Phase A, FFI scaffolding
12:32	<code>xrpl-validator</code>	<code>8e7a6de</code>	feat(ffi): first real libxrpl call — parse mainnet Payment tx
12:39	<code>xrpl-validator</code>	<code>9ac92d5</code>	feat(ffi): first successful <code>xrpl::preflight()</code> call on real mainnet tx
12:46	<code>xrpl-validator</code>	<code>2c9c10e</code>	feat(ffi): <code>xrpl::apply()</code> SUCCESS on real mainnet Payment
12:49	<code>xrpl-validator</code>	<code>2ddeefe</code>	feat(ffi): live demo script for visible FFI pipeline
12:53	<code>xrpl-validator</code>	<code>a76625e</code>	docs(ffi): clearer demo — status banner + honest labels
13:01	<code>xrpl-validator</code>	<code>67eb437</code>	feat(ffi): extract state mutations after <code>apply()</code> — full round trip
13:08	<code>xrpl-validator</code>	<code>eee986d</code>	feat(ffi): Rust FFI bindings — crate <code>xrpl-ffi</code> with full round trip tests
13:21	<code>xrpl-validator</code>	<code>d8da1f8</code>	feat(ffi): integrate <code>xrpl-ffi</code> into validator (feature-gated)
13:25	<code>xrpl-validator</code>	<code>1d7e2eb</code>	feat(ffi): push live mainnet tx through libxrpl in real-time
13:30	<code>xrpl-validator</code>	<code>7d0acd7</code>	feat(ffi): full <code>apply()</code> on live mainnet tx with RPC-fetched state
15:06	<code>xrpl-validator</code>	<code>0f7682f</code>	feat(ffi): 100% mainnet agreement — libxrpl apply in full
15:24	<code>xrpl-validator</code>	<code>00b54ca</code>	feat(ffi): production observability — Prometheus, Grafana, SLOs
17:36	<code>xrpl-validator</code>	<code>9993061</code>	fix(ffi): retry + failover against rippled overload
17:53	<code>xrpl-validator</code>	<code>5c0eef0</code>	refactor(validator): <code>tx_engine</code> → <code>FfiVerifier</code> (libxrpl via FFI)
18:00	<code>xrpl-validator</code>	<code>b1aad92</code>	feat(ffi): Stage 1b — RocksDB-backed SLE provider
18:45	<code>xrpl-validator</code>	<code>0b275be</code>	fix(ffi): <code>OwnedSnapshot</code> eliminates verify race condition
19:06	<code>xrpl-validator</code>	<code>8d1d941</code>	fix(sync): <code>bulk_sync rpc_failover</code> was ignoring <code>XRPL_RPC_URL</code> override
19:11	<code>xrpl-validator</code>	<code>c2f8f6a</code>	fix(ffi): disable DB snapshot during catchup — use pure RPC path
19:18	<code>xrpl-validator</code>	<code>1fc4d55</code>	fix(rpc): <code>rpc_request()</code> was ignoring <code>XRPL_RPC_URL</code> env override — used hardcoded localhost
19:41	<code>xrpl-validator</code>	<code>e21113b</code>	refactor(dashboard): update <code>watch_engine.py</code> for integrated FFI validator
20:28	<code>xrpl-validator</code>	<code>9613958</code>	feat(ui): new validator dashboard designed with Google Stitch
20:32	<code>xrpl-validator</code>	<code>6bcb8ff</code>	feat(ui): wire live data into dashboard — 500ms fetch from <code>/api/engine</code>
20:44	<code>xrpl-validator</code>	<code>80eb182</code>	feat(ui): live tx-type bars with color-coded fills replacing static chart
20:54	<code>xrpl-validator</code>	<code>afdbc36</code>	feat(ui): per-ledger tx bars (reset each ledger) + sync progress indicator
20:57	<code>xrpl-validator</code>	<code>ec154d0</code>	fix(ui): remove <code>/api/status</code> dependency, infer sync from FFI state
20:59	<code>xrpl-validator</code>	<code>12b4bf8</code>	feat(ui): show all tx types grayed out, light up when active in current ledger
21:05	<code>xrpl-validator</code>	<code>c190092</code>	feat(ui): state lifecycle indicator (sync → build → match) + live object count + round tx stats

Time	Repo	Commit	Subject
21:07	xrpl-validator	6f21057	feat(ui): descriptive labels + explanations for FFI engine stats
21:12	xrpl-validator	77082a0	feat(ui): add WIP notice — dashboard in development, not all links functional